

LAPORAN 4

ANALISIS KOMPLEKSITAS ALGORITMA

Disusun untuk memenuhi tugas laporan



DISUSUN OLEH :

ADILLA SALSABILA
NIM 2025573010106

Kelas : TI 1C
Mata Kuliah : Algoritma dan struktur data
Jurusan : Teknologi Informasi dan Komputer
Prodi : Teknik Informatika
Dosen : M. REZA ZULMAN

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2025-2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT, Tuhan Yang Maha Esa atas rahmat dan karunia- Nya, sehingga penulis dapat menyelesaikan laporan ini . Mata kuliah ini dirancang untuk membahas penggunaan bahasa pemograman JavaScript dalam logika algoritma. Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun dari para pembaca agar laporan ini dapat lebih baik di masa mendatang.

Semoga laporan ini dapat bermanfaat bagi para pembaca dan dapat memberikan sumbangsih dalam bidang pengembangan teknologi informasi dan transformasi digital.

Lhokseumawe, 28 april 2026

Penulis

Adilla Salsabila

DAFTAR ISI

DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Tujuan Penulisan	4
1.3 Manfaat Penulisan	4
BAB II PEMBAHASAN	6
2.1 Alur kerja dan latihan pemograman kompleksitas algoritma	6
2.2 Tugas Pratikum	6
BAB III PENUTUP	9
3.1 Kesimpulan.....	9
3.2 Saran	9

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman modern tidak hanya menuntut kode yang fungsional, tetapi juga efisien secara struktur dan pengelolaan. JavaScript menyediakan berbagai fitur seperti *Function* dan *Array Methods* yang menjadi fondasi penting dalam algoritma dan struktur data. Melalui analisis kompleksitas, pengembang dapat membedakan mana algoritma yang mampu menangani data skala besar secara optimal dan mana yang akan mengalami penurunan performa.

1.2 Tujuan Penulisan

- Memahami Notasi Big O: Mampu mengidentifikasi kelas kompleksitas waktu mulai dari $O(1)$, $O(n)$, hingga $O(2^n)$.
- Analisis Rekursi dan Iterasi: Memahami penerapan teknik rekursi serta teknik optimasi seperti *memoization* untuk menyelesaikan masalah matematis.
- Pengukuran Performa: Mampu mengukur waktu eksekusi fungsi secara akurat menggunakan `Date.now()` atau `performance.now()`.
-

1.3 Manfaat Penulisan

- Peningkatan Keterampilan Teknis: Mengasah kemampuan menulis kode JavaScript yang bersih dan efisien melalui teknik *refactoring*.
- Bukti Autentik Praktikum: Sebagai dokumentasi resmi atas pemahaman materi yang telah disampaikan oleh dosen pengampu.

BAB II

PEMBAHASAN

2.1 Alur kerja dan latihan pemograman kompleksitas algoritma

Buka VSCode dan pilih folder hasil yang sudah di *clone*. Kemudian, buka terminal, pindah ke opsi **Git Bash**, dan buat folder baru. Lalu buat file baru di dalam folder tsbt.



- File : 01-intro-kompleksitas.js

```
JS 01-intro-kompleksitas.js > ...
1  function ukurWaktu(label, fn) {
2      const mulai = Date.now();
3      fn();
4      const selesai = Date.now();
5      console.log(`${label}: ${selesai - mulai} ms`);
6  }
7  const N = 100_000;
8  function jumlahkanLinear(n) {
9      let total = 0;
10     for (let i = 1; i <= n; i++) total += i;
11     return total;
12 }
13 function jumlahkanRumus(n) {
14     return (n * (n + 1)) / 2;
15 }
16 function cariPasangan(arr) {
17     const pasangan = [];
18     for (let i = 0; i < arr.length; i++) {
19         for (let j = i + 1; j < arr.length; j++) {
20             if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
21         }
22     }
23     return pasangan;
24 }
25 const data = Array.from({ length: 5000 }, (_, i) => i);
26 console.log("=== Perbandingan Waktu Eksekusi ===");
27 ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
28 ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
29 ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));
30 console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

Hasil :

```
USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 2 ms
O(n2) cariPasangan : 40 ms

Hasil sama? true
```

- File : 02-kelas-big-o.js

```
JS 02-kelas-big-o.js > ...
1  function ambilPertama(arr) {
2  |   return arr[0];
3  | }
4  function tambahkanItem(arr, item) {
5  |   arr.push(item);
6  | }
7  function isGenap(n) {
8  |   return n % 2 === 0;
9  | }
10
11 function binarySearch(arr, target) {
12 |   let kiri = 0,
13 |   |   kanan = arr.length - 1;
14 |   let langkah = 0;
15 |   while (kiri <= kanan) {
16 |   |   langkah++;
17 |   |   const tengah = Math.floor((kiri + kanan) / 2);
18 |   |   if (arr[tengah] === target) {
19 |   |   |   console.log(" Ditemukan di indeks ${tengah} setelah ${langkah} langkah");
20 |   |   |   return tengah;
21 |   |   }
22 |   |   if (arr[tengah] < target) kiri = tengah + 1;
23 |   | }
24 |   return -1;
25 | }
26 function cariMax(arr) {
27 |   let maks = arr[0];
28 |   for (const x of arr) if (x > maks) maks = x;
29 |   return maks;
30 | }
31 function bubbleSortNaif(arr) {
32 |   const a = [...arr];
33 |   for (let i = 0; i < a.length; i++) {
34 |   |   // stop earlier each pass (last i elements already sorted)
35 |   |   for (let j = 0; j < a.length - i - 1; j++) {
36 |   |   |   if (a[j] > a[j + 1]) {
37 |   |   |   |   [a[j], a[j + 1]] = [a[j + 1], a[j]];
38 |   |   |   }
39 |   |   }
40 |   }
41 |   return a;
42 | }
43 function fibRekursif(n) {
44 |   if (n <= 1) return n;
45 |   return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
46 | }
47 console.log("\n=== O(1) - selalu cepat ===");
48 console.log(ambilPertama([10, 20, 30, 40, 50])); //10
49 console.log(isGenap(42));
50
51 console.log("\n=== O(log n) - Binary search ===");
52 const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
53 binarySearch(sorted, 731_452);
54
55 console.log("\n=== O(n) - Linear Search ===");
56 console.log(
57 |   "Max dari 1000 elemen:",
58 |   cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
59 | );
60
61 console.log("\n=== O(n^2) - Bubble Sort (kecil saja) ===");
62 console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
```

```

64 console.log("\n=== O(2n) - Fibonacci Rekursif (n kecil!) ===");
65 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
66 console.log("");
67
68 console.log("\nWaktu fib(35) O(2n):");
69 let t = Date.now();
70 fibRekursif(35);
71 console.log(Date.now() - t, "ms");
72 console.log("Waktu fib(35) O(n) memoization:");
73 const memo = {};
74 function fibMemo(n) {
75   if (n <= 1) return n;
76   if (memo[n]) return memo[n];
77   return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
78 }
79 t = Date.now();
80 fibMemo(35);
81 console.log(Date.now() - t, "ms");
82

```

Hasil :

```

USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 02-kelas-big-o

=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary search ===

=== O(n) - Linear Search ===
Max dari 1000 elemen: 995

=== O(n2) - Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) - Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
274 ms
Waktu fib(35) O(n) memoization:
0 ms

```

- latihan 1 :

```
JS latihan-1.js > fungsiA
1 function fungsiA(n) {
2   return n * 2;
3 }
4 function fungsiB(n) {
5   for (let i = 0; i < n; i++) {
6     for (let j = 0; j < n; j++) {}
7   }
8 }
9 function fungsiC(n) {
10  for (let i = 1; i < n; i *= 2) {}
11 }
12 function fungsiD(n) {
13   const arr = Array.from({ length: n }, (_, i) => i);
14
15   arr.forEach((x) => {
16     arr.forEach((y) => {
17       arr.forEach((z) => {});
18     });
19   });
20 }
21 function hitungKompleksitas(n, fn, namaFungsi) {
22   const start = Date.now();
23   fn(n);
24   const end = Date.now();
25   console.log(`Waktu eksekusi ${namaFungsi} (n=${n}): ${end - start} ms`);
26 }
27
28 const n = 1000;
29 console.log("--- Pengukuran Waktu Eksekusi ---");
30 hitungKompleksitas(n, fungsiA, "Fungsi A [O(1)]");
31 hitungKompleksitas(n, fungsiB, "Fungsi B [O(n^2)]");
32 hitungKompleksitas(n, fungsiC, "Fungsi C [O(log n)]");
33 hitungKompleksitas(n, fungsiD, "Fungsi D [O(n^3)]");
```

Hasil :

```
USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5 (main)
$ node latihan-1.js
--- Pengukuran Waktu Eksekusi ---
Waktu eksekusi Fungsi A [O(1)] (n=1000): 0 ms
Waktu eksekusi Fungsi B [O(n^2)] (n=1000): 4 ms
Waktu eksekusi Fungsi C [O(log n)] (n=1000): 0 ms
Waktu eksekusi Fungsi D [O(n^3)] (n=1000): 1203 ms
```


- File : 03-space-complexity.js

```
JS 03-space-complexity.js > ...
1  function jumlahArray(arr) {
2    let total = 0; // hanya 1 variabel tambahan
3    for (const x of arr) total += x;
4    return total;
5  }
6  function duplikasiArray(arr) {
7    const baru = [];
8    for (const x of arr) baru.push(x * 2);
9    return baru;
10 }
11 function faktorialRekursif(n) {
12   if (n <= 1) return 1;
13   return n * faktorialRekursif(n - 1);
14 }
15 function faktorialIteratif(n) {
16   let hasil = 1;
17   for (let i = 2; i <= n; i++) hasil *= i;
18 }
19 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
20
21 console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
22 console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
23 console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
24 console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
25 function hitungUnik(arr) {
26   const seen = new Set();
27   for (const x of arr) seen.add(x);
28   return seen.size;
29 }
30 const dataAcak = [1, 2, 3, 4, 5, 6, 7];
31 console.log("Elemen unik:", hitungUnik(dataAcak)); // 7
```

Hasil :

```
USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : undefined
Elemen unik: 7
```

latihan 3 :

```
//Latihan 3: Analisis Time & Space Complexity

// 2a. cariPasanganLambat (Nested Loop)
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// 2b. cariPasanganCepat (Menggunakan Set)
function cariPasanganCepat(arr, target) {
  const seen = new Set();
  for (let num of arr) {
    let complement = target - num;
    if (seen.has(complement)) {
      return [complement, num];
    }
    seen.add(num);
  }
  return null;
}

console.log('');
console.log ('== latihan 2==');

// 3. Uji fungsi dengan data [2, 7, 11, 15] dan target 9
const testArray = [2, 7, 11, 15];
const target = 9;

console.log("Hasil Uji Sederhana:");
console.log("Lambat:", cariPasanganLambat(testArray, target)); // [2, 7]
console.log("Cepat:", cariPasanganCepat(testArray, target)); // [2, 7]

const besarArray = 50000;
const dataAcak = Array.from({ length: besarArray }, () => Math.floor(Math.random() * 100000));
const targetSulit = -1;

console.log("\n--- Pengukuran Waktu (50.000 data) ---");

const startLambat = Date.now();
cariPasanganLambat(dataAcak, targetSulit);
const endLambat = Date.now();
console.log(`Waktu cariPasanganLambat: ${endLambat - startLambat} ms`);

const startCepat = Date.now();
cariPasanganCepat(dataAcak, targetSulit);
const endCepat = Date.now();
console.log(`Waktu cariPasanganCepat: ${endCepat - startCepat} ms`);
```

Hasil :

```
== latihan 2==
Hasil Uji Sederhana:
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]

--- Pengukuran Waktu (50.000 data) ---
Waktu cariPasanganLambat: 4287 ms
Waktu cariPasanganCepat: 9 ms
```

- file : 04-refactor.js

```
JS 04-refactor.js > ...
1  function adaDuplikatLambat(arr) {
2    for (let i = 0; i < arr.length; i++)
3      for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
4    return false;
5  }
6  function adaDuplikatCepat(arr) {
7    const seen = new Set();
8    for (const x of arr) {
9      if (seen.has(x)) return true;
10     seen.add(x);
11   }
12   return false;
13 }
14 function frekuensiLambat(arr) {
15   const unik = [];
16   const hitung = [];
17   for (const x of arr) {
18     const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
19     if (idx === -1) {
20       unik.push(x);
21       hitung.push(1);
22     } else hitung[idx]++;
23   }
24   return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
25 }
26 function frekuensiCepat(arr) {
27   const counter = {};
28   for (const x of arr) counter[x] = (counter[x] || 0) + 1;
29   return counter;
30 }
31 const besar = Array.from({ length: 20000 }, () =>
32   Math.floor(Math.random() * 1000),
```

Hasil :

```
USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

2.2 Tugas Pratikum

Tugas-1 : analisis refactor

```
ugas > JS Tugas-1 > checkTripletFast
1 function intersectionSlow(arr1, arr2) {
2   |   return arr1.filter(item => arr2.includes(item));
3 }
4 function intersectionFast(arr1, arr2) {
5   |   const set1 = new Set(arr1);
6   |   return arr2.filter(item => set1.has(item));
7 }
8 function groupAnagrams(strs) {
9   |   const map = new Map();
10  |   for (let s of strs) {
11  |     |
12  |     |   const key = s.split('').sort().join('');
13  |     |   if (!map.has(key)) map.set(key, []);
14  |     |   map.get(key).push(s);
15  |     |
16  |   }
17  |   return Array.from(map.values());
18 }
19 function checkTripletSlow(arr) {
20   |   for (let i = 0; i < arr.length; i++) {
21   |     |   for (let j = i + 1; j < arr.length; j++) {
22   |     |     |   for (let k = 0; k < arr.length; k++) {
23   |     |     |     |   if (arr[i] + arr[j] === arr[k] * arr[k]) return true;
24   |     |     |     |
25   |     |     |   }
26   |     |   }
27   |   }
28   |   return false;
29 }
30 function checkTripletFast(arr) {
31   |   const sorted = [...arr].sort((a, b) => a - b);
32   |   const squares = new Set(arr.map(x => x * x));
33   |   for (let i = 0; i < sorted.length; i++) {
34   |     |   for (let j = i + 1; j < sorted.length; j++) {
35   |     |     |   let sum = sorted[i] + sorted[j];
36   |     |     |   if (squares.has(sum)) return true;
37   |     |     |
38   |     |   }
39   |   }
40   |   return false;
41 }
42 function benchmark(label, fn, ...args) {
43   |   const start = performance.now();
44   |   const result = fn(...args);
45   |   const end = performance.now();
46   |   console.log(`${label.padEnd(25)} | Hasil: ${result.toString().slice(0,10)} | Waktu: ${(end - start).toFixed(4)} ms`);
47 }
48 const bigArr1 = Array.from({ length: 10000 }, (_, i) => i);
49 const bigArr2 = Array.from({ length: 10000 }, (_, i) => i + 5000);
50 const anagrams = ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'];
51 const triplets = [3, 1, 4, 6, 5]; // 4 + 5 = 3^2
52 console.log("--- Menjalankan Tugas 1 ---");
53 console.log("\n[Fungsi A: Intersection]");
54 benchmark("Intersection O(n^2)", intersectionSlow, bigArr1, bigArr2);
55 benchmark("Intersection O(n)", intersectionFast, bigArr1, bigArr2);
56 console.log("\n[Fungsi B: Anagram]");
57 console.log("Input:", anagrams);
58 console.log("Output:", JSON.stringify(groupAnagrams(anagrams)));
59 console.log("\n[Fungsi C: Triplet a+b=c^2]");
60 const mediumArr = Array.from({ length: 500 }, (_, i) => i + 1);
61 benchmark("Triplet O(n^3)", checkTripletSlow, mediumArr);
62 benchmark("Triplet O(n log n / n^2)", checkTripletFast, mediumArr);
```

Hasil run :

```
USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5/tugas (main)
$ node Tugas-1
--- Menjalankan Tugas 1 ---

[Fungsi A: Intersection]
Intersection  $O(n^2)$       | Hasil: 5000,5001, | Waktu: 107.0158 ms
Intersection  $O(n)$        | Hasil: 5000,5001, | Waktu: 3.5779 ms

[Fungsi B: Anagram]
Input: [ 'eat', 'tea', 'tan', 'ate', 'nat', 'bat' ]
Output: [ ["eat","tea","ate"], ["tan","nat"], ["bat"] ]

[Fungsi C: Triplet  $a+b=c^2$ ]
Triplet  $O(n^3)$            | Hasil: true | Waktu: 0.1243 ms
Triplet  $O(n \log n / n^2)$  | Hasil: true | Waktu: 0.2543 ms
```

Tugas-2 : visualisasi pertumbuhan big o

```
ugas > JS Tugas-2 > ...
1  function fn_O1(n) {
2    |   return n + 1;
3  }
4  function fn_On(n) {
5    |   let sum = 0;
6    |   for (let i = 0; i < n; i++) {
7    |     |   sum += i;
8    |     |
9    |     return sum;
10  }
11 function fn_OnLogn(n) {
12   |   let count = 0;
13   |   for (let i = 0; i < n; i++) {
14   |     |   for (let j = 1; j < n; j *= 2) {
15   |     |     |   count++;
16   |     |     |
17   |     |     }
18   |     |
19   |     return count;
20  }
21 function fn_On2(n) {
22   |   let count = 0;
23   |   for (let i = 0; i < n; i++) {
24   |     |   for (let j = 0; j < n; j++) {
25   |     |     |   count++;
26   |     |     |
27   |     |     }
28   |     |
29   |     return count;
30  }
31 function benchmarkSemua(ukuranData) {
32   |   console.log("Ukuran Data | O(1) (ms) | O(n) (ms) | O(n log n) (ms) | O(n^2) (ms)");
33   |   console.log("-----");
34   |
35   |   ukuranData.forEach(n => {
36   |     |   const t0 = performance.now();
37   |     |   fn_O1(n);
38   |     |   const d1 = (performance.now() - t0).toFixed(4);
39   |     |
40   |     |   const t1 = performance.now();
41   |     |   fn_On(n);
42   |     |   const dn = (performance.now() - t1).toFixed(4);
43   |     |
44   |     |   const t2 = performance.now();
45   |     |   fn_OnLogn(n);
46   |     |   const dnlog = (performance.now() - t2).toFixed(4);
47   |     |
48   |     |   const t3 = performance.now();
49   |     |   fn_On2(n);
50   |     |   const dn2 = (performance.now() - t3).toFixed(4);
51   |     |
52   |     |   console.log(`${n.toString().padEnd(11)} | ${d1.padEnd(9)} | ${dn.padEnd(9)} | ${dnlog.padEnd(15)} | ${dn2}`);
53   |     |   });
54   |   }
55   benchmarkSemua([100, 500, 1000, 5000, 10000]);
56 }
```

Hasil :

```
USER@USER-PC MINGW64 /d/2025573010106_algoritma_dan_struktur_data/praktikum-5/tugas (main)
$ node tugas-2
Ukuran Data | O(1) (ms) | O(n) (ms) | O(n log n) (ms) | O(n^2) (ms)
-----
100          | 0.0185    | 0.0365    | 0.0493          | 0.3374
500          | 0.0031    | 0.0091    | 0.1072          | 1.0794
1000         | 0.0020    | 0.0575    | 0.1045          | 2.9417
5000         | 0.0019    | 0.0121    | 0.1558          | 64.1897
10000        | 0.0133    | 0.0341    | 0.6206          | 223.8908
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan beberapa poin utama:

- Efisiensi Algoritma: Penggunaan struktur data yang tepat, seperti *Set* untuk pencarian pasangan, terbukti jauh lebih cepat ($O(n)$) dibandingkan dengan metode *Nested Loop* ($O(n^2)$) saat menangani data dalam jumlah besar.
- Optimasi Rekursi: Teknik rekursi memberikan solusi logis yang efektif, namun harus digunakan dengan bijak. Penerapan *memoization* secara signifikan dapat memangkas waktu eksekusi pada algoritma seperti Fibonacci dari kompleksitas eksponensial menjadi linear.
- Modularitas Kode: Penguasaan *arrow functions*, *callback*, serta metode iterasi modern (*map*, *filter*, *reduce*) mendukung terciptanya kode yang lebih ringkas, mudah dipelihara, dan modular.

• 3.2 Saran

- Manajemen Repositori: Praktikan disarankan untuk selalu melakukan *push* seluruh file latihan ke GitHub sebagai bentuk cadangan data yang aman.
- Kemandirian Debugging: Praktikan sebaiknya lebih aktif memanfaatkan pesan kesalahan (*error message*) di terminal Node.js untuk belajar mengidentifikasi serta memperbaiki kerusakan kode secara mandiri.